1 Exercise 6: Timer/Counter and PWM (Microchip Studio) (MEGA)

1.1 Useful References

- ATmega2560 Datasheet
- AVR LIBC manual

1.2 Overview

In this task, use PWM and the 16-bit timer/counter to produce notes with the passive buzzer.

1.3 Building Circuit

Build the circuit using these components:

- Arduino Mega (Mega 2560)
- 2 x Resistor 220 Ohm
- Diode 1N4007
- LED
- Passive Buzzer (One without a sticker).
- Wires

The schematic of the circuit is as follows.

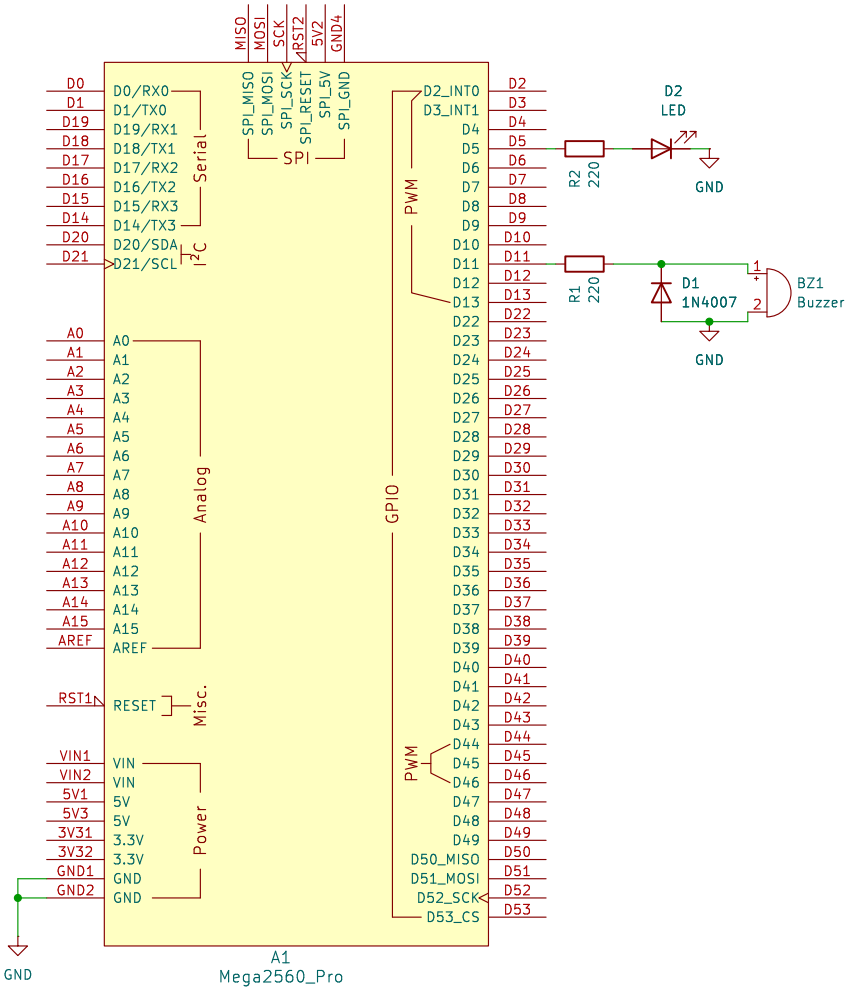

Figure 1: Circuit schematic.

- Connect positive leg of the passive buzzer to Arduino Digital IO pin 11 with 220 Ohm current-limiting resistor.
- Connect positive leg of the passive buzzer to the cathode (-) of 1N4007 diode.
- Connect anode of the diode and negative leg of the buzzer to ground.
- Connect anode of LED to Arduino Digital IO pin 5 with 220 Ohm current-limiting resistor.
- Connect cathode of LED to ground.
- Connect any ground pin of Mega board to your circuit's ground bar.

The diode here is used as a flyback diode to protect the microcontroller from overvoltages when the inductive load (magnetic buzzer) is disconnected.

1.4 Included Code

Create main.c file in your project. Include the following provided headers and libraries in your project and add the required `#include` directives for header files to main.c unit:

- mcu.h for microcontroller-related defines
- `bit_ops.h` for bit manipulation in registers from solution to Exercise 5
- `board_config_2560.h` for Arduino Mega board pin definitions from solution to Exercise 3
- critical.h and critical.c for critical sections implemented in Exercise 5
- timer0.h and timer0.c for precise timer implemented in Exercise 5
- uart.h and uart.c for serial port driver implemented in Exercise 1

1.5 Clearing bitops

Edit `bit_ops.h` to add new macro that will allow clearing (setting to 0) of multibit fields. Use `SET_BITS` from exercise 5 as the example.

```
#define CLEAR_BITS(val, bit, mask) Implement me
```

1.6 Timer 1 driver

Create and add to project `timer1.h` Add to it the following definitions

```
#ifndef TIMER1_H
#define TIMER1_H

#include <stdint.h>

/// Set up the 8-bit timer/counter1.
/// This function configures timer1 to generate PWM of fixed
/// 50% duty cycle and varying frequency.
void setup_timer1(void);

/// Disables PWM output on channel A pin
void timer1_channel_A_off(void);

/// Enables PWM output on channel A pin
void timer1_channel_A_on(void);

/// Changes prescaler and conparison register
/// to produce the desired frequency of PWM signal
void timer1_set_frequency(uint16_t frequency_hz);

#endif //TIMER1_H
```

Then create and add to project `timer1.c` and copy the definitions (without the guard section) there. Also add the following includes

```
#include "timer1.h"
#include "mcu.h"
#include <avr/interrupt.h>
#include <avr/io.h>
#include <stdio.h>

#include "bit_ops.h"
```

Add and fill (replace exclamation marks) the following declarations using the datasheet. Use timer0.c as an example

```
/// Datasheet p. 154
#define TCCR1A_WGM_MODE_OFFSET !!!
#define TCCR1A_WGM_MODE_MASK !!!
#define TCCR1A_COMPARE_MODE_CH_A_OFFSET !!!
#define TCCR1A_COMPARE_MODE_CH_A_MASK !!!

#define TCCR1A_COMPARE_MODE_NORMAL_IO !!!
#define TCCR1A_COMPARE_MODE_TOGGLE_ON_MATCH !!!


/// Datasheet p. 145 Table 17-2
        #define WGM_MODE_PWM_PHASE_FREQ_CORRECT_OCRA (9)

/// Datasheet p. 156
#define TCCR1B_CLOCK_SELECT_OFFSET !!!
#define TCCR1B_CLOCK_SELECT_MASK !!!
#define TCCR1B_WGM_MODE_OFFSET !!!
#define TCCR1B_WGM_MODE_MASK !!!

/// Human-readable registers
#define CONTROL_REGISTER_A (TCCR1A)
#define CONTROL_REGISTER_B (TCCR1B)
#define TIMER_COUNT (TCNT1)
#define OUTPUT_COMPARE_A (OCR1A)
#define OUTPUT_COMPARE_B (OCR1B)
#define INTERRUPT_MASK (TIMSK1)
#define INTERRUPT_FLAGS (TIFR1)
```

Declare `timer1_prescaler_t` enumerable with all possible prescaler options mapped to their values from datasheet Table 17-6 p. 157 and a lookup table `PRESCALER_DIVIDERS` with inverse mapping from `timer1_prescaler_t` to the actual scaling factor.

```
typedef enum timer1_prescaler {
    TIMER1_CLOCK_OFF = 0,
    TIMER1_PRESCALER_1 = 1,
    TIMER1_PRESCALER_8 = 2,
    TIMER1_PRESCALER_64 = 3,
    TIMER1_PRESCALER_256 = 4,
    TIMER1_PRESCALER_1024 = 5,
    TIMER1_EXTERNAL_CLOCK_FALLING = 6,
    TIMER1_EXTERNAL_CLOCK_RAISING = 7,
} timer1_prescaler_t;

const uint16_t PRESCALER_DIVIDERS[] = {1, 1, 8, 64, 256, 1024, 1, 1};
```

Implement `set_waveform_generation_mode` helper following the similar function in timer0 as an example.

Implement module's public interfaceis.

`timer1_channel_A_off` and `timer1_channel_A_on` modify (clear then set new value) `COMPARE_MODE_CH_A` bits of `CONTROL_REGISTER_A` to either `TCCR1A_COMPARE_MODE_NORMAL_IO` (off) or `TCCR1A_COMPARE_MODE_TOGGLE_ON` (on)

`timer1_set_frequency` modifies `CLOCK_SELECT` bits of `CONTROL_REGISTER_B` to select a prescaler, computes value for register `OUTPUT_COMPARE_A` using the desired frequency, cpu frequency and prescaler value. Prescaler 64 should be used for frequencies below 8 Hz, prescaler 8 for frequencies betweenn 8 and 64 Hz and no prescaler should be used for frequencies above 64 Hz. The following snippet demonstrates `OUTPUT_COMPARE_A` value calculation for prescaler selected in `timer1_prescaler_t prescaler` variable.

```
// Lookup the actual divider

const uint16_t divider = PRESCALER_DIVIDERS[prescaler];

/// Extra division by 4 comes from 2 factors:
/// 1) In phase and frequency correct PWM mode the counter counts up to OCRA and then back to 0 so it do
/// the effective period to 2 * OCRA
/// 2) The output is toggled once every time the timer reaches OCRA. Therefore it takes two full countin
/// toggle low to high and then high to low.
const uint32_t ticks_per_s = ((uint32_t) F_CPU) / divider / 4;

OUTPUT_COMPARE_A = ticks_per_s / frequency_hz;
```

Implement a reset function zeroing all timer1 registers similar to one in timer0 driver.

Finally implement the public interface `setup_timer0` which should:

- Reset module
- Set waveform generation mode to `WGM_MODE_PWM_PHASE_FREQ_CORRECT_OCRA`
- Turn output on channel A off.
- Print out all register values to serial as hexadecimals (`%x`) one on each line to allow diagnosing of configuration issues.

1.7 Tune helper header

Create and add to project tune.h with the following content.

```
#ifndef TUNE_H
#define TUNE_H

#define BPM (90)
#define FULL (60000/BPM)
#define HALF (FULL/2)
#define QUARTER (FULL/4)

#define C3 262
#define D3 293
#define E3 330
#define F3 349
#define G3 392
#define A3 440
#define B3 494
#define C4 523

typedef struct note {
        uint16_t frequency_hz;
        uint16_t duration_ms;
} note_t;
#endif
```

Here we declare beats per minute rate of our tune and define durations of notes as well as come frequencies for C-major scale. Finally we implement a struct that will store the frequancy of the note and its duration.

1.8 Continuing main.c

Return to main.c and include just created timer1.h and the following library headers

```
#include "mcu.h"
#include "uart.h"
#include "board_config_2560.h"
#include "timer0.h"
#include "timer1.h"
#include "tune.h"

#include <util/delay.h>
#include <string.h>
#include <stdio.h>
#include <avr/interrupt.h>
#include <avr/sleep.h>
#include <stdbool.h>
```

1.9 Initialization

Write `handle_error` function that uses LED to inidicate the error code if non 0 with a blink patter or a short blink for 0 errors.

```
static void handle_error(uint8_t return_code)
{
        CLEAR_BIT(IO_5_PORT, IO_5_PIN);
        SET_BIT(IO_5_DIRECTION, IO_5_PIN);
        // Non-zero return code indicates critical fault
        if (return_code)
        {
                while(1) {
                        // Long blink, then short blinks based on the return_code
                        // Repeat to keep indicating fault
                        _delay_ms(1000);
                        CLEAR_BIT(IO_5_PORT, IO_5_PIN);
                        _delay_ms(200);
                        for (uint8_t i = 0; i < return_code; i++) {
                                SET_BIT(IO_5_PORT, IO_5_PIN);
                                _delay_ms(200);
                                CLEAR_BIT(IO_5_PORT, IO_5_PIN);
                                _delay_ms(200);
                        }
                }
        }

        // Short blink as confirmation for everything working fine.
        SET_BIT(IO_5_PORT, IO_5_PIN);
        _delay_ms(100);
        CLEAR_BIT(IO_5_PORT, IO_5_PIN);
}
```

Then write a `void setup(void)` function that implements the following initialization steps.

- Initialize serial port for standard library. Handle possible errors with `handle_error`
- Set Port C pin 11 as output with default output low.
- Call setup function of the Timer 0

- Call setup function of the Timer 1
- Set sleep mode of processor with `set_sleep_mode(SLEEP_MODE_IDLE);`
- Enable interrupts globally with `sei();`
- Write 'Ready' to serial port
- Busy-wait 1 second

Log progress of initialization to serial port starting right after calling `setup_uart_io();` and checking its exit code.

Implement sleep helper that waits for wall clock time to advance by the provided value of milliseconds.

```
static void sleep_ms(uint32_t time) {
        uint32_t end = get_time() + time/10;
        while (get_time() < end) {
                sleep_mode();
        }
}
```

1.10 Sample tune data

Declare arrays containing sample tunes. One is a simple sweep across different frequencies and the other is a simple tune with arpeggiated chords.

```
note_t test_tune_1[] = {
        { 3, HALF },
        { 15, HALF },
        { 50, HALF },
        { 100, HALF },
        { 150, HALF },
        { 300, HALF },
        { 400, HALF },
        { 600, HALF },
        { 600, HALF }};

note_t test_tune_2[] = {
        { C3, QUARTER },
        { E3, QUARTER },
        { G3, QUARTER },
        { C3, QUARTER },
        { E3, QUARTER },
        { G3, QUARTER },
        { E3, QUARTER },
        { G3, QUARTER },
        { B3, QUARTER },
        { E3, QUARTER },
        { G3, QUARTER },
        { B3, QUARTER },
        { A3, QUARTER },
        { F3, QUARTER },
        { D3, QUARTER },
        { A3, QUARTER },
        { F3, QUARTER },
```

1.11 Main function and loop

Start the main function with a call to setup and declaration of necessary variables, e.g. an index of current note in the tune.

In the while loop:

- Check if current index is within the size of the melody. If not - reset to 0 and skip the loop. One way to get the size of array is using sizeof fubction as `sizeof(array_name)/sizeof(array_element_t)`
- Log the next note to play to serial port for debugging.
- Call `timer1_set_frequency` to update the played note.
- Turn PWM output on.
- Call `sleep_ms` to wait for note duration.
- Turn PWM output off.
- Increment pointer to.the next note to play.